Databricks SQL Warehouse

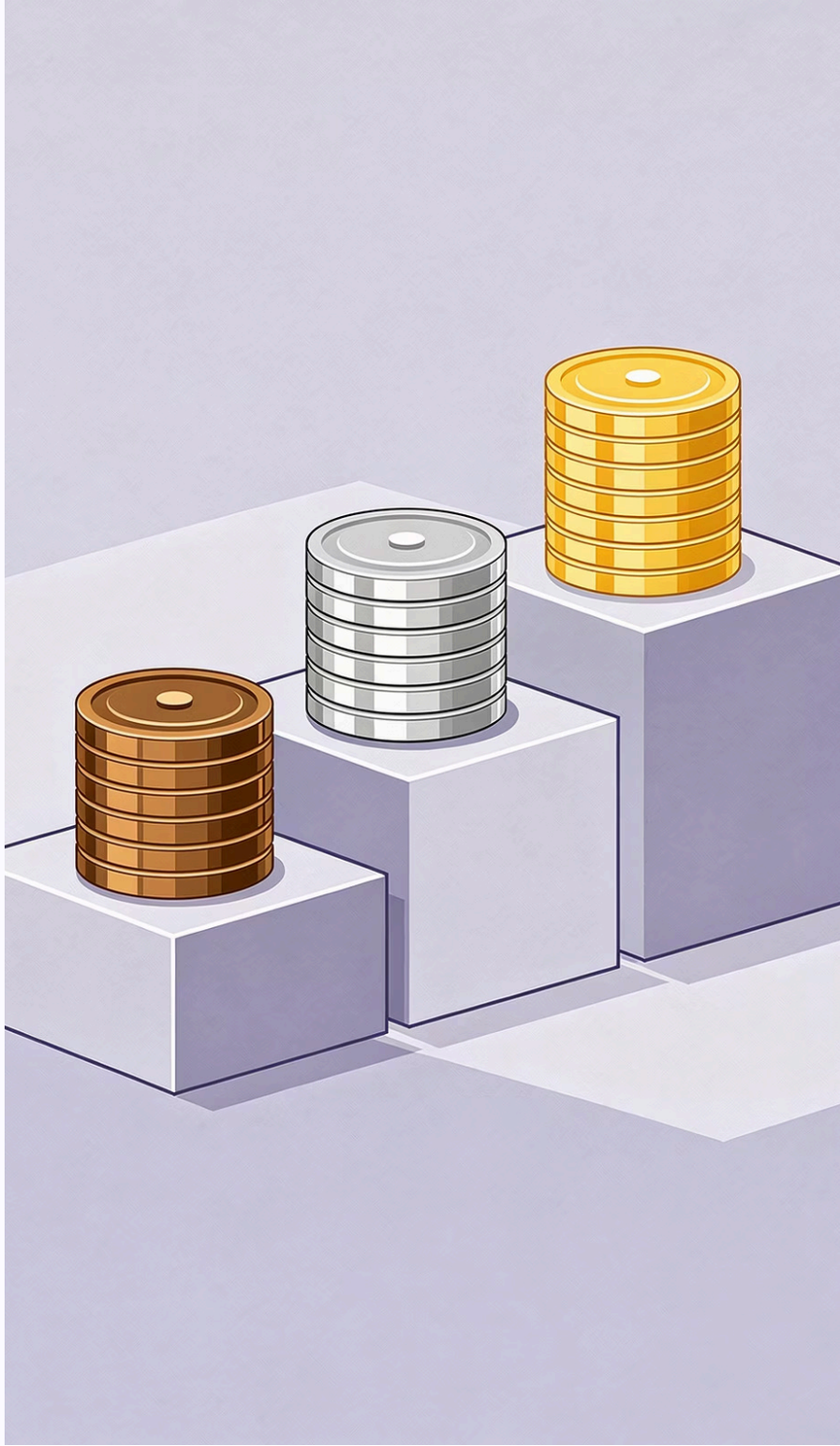
Serving the Gold Layer to the Business

Where We Left Off

We have built a fully structured **Medallion Architecture** — ingesting raw data into **Bronze**, cleansing and conforming it into **Silver**, and aggregating business-ready metrics into **Gold**. Each layer serves a distinct purpose in your data pipeline.

> The Gold layer holds curated, high-value data — but it's only useful if the right people can query it efficiently. Today's question: **How is Gold actually consumed?**

1. **Bronze**
   Raw ingestion, immutable records
2. **Silver**
   Cleaned, conformed data
3. **Gold**
   Business-ready aggregates
4. **SQL Warehouse**
   Serving layer for consumers



What Is a SQL Warehouse?

Definition

A SQL Warehouse is Databricks' **dedicated, serverless SQL execution engine** — purpose-built for running analytical queries against Delta tables at scale. Unlike general-purpose clusters, it is fully managed and requires zero infrastructure configuration from the analyst.

Core Characteristics

Serverless SQL Engine

No cluster configuration required. Compute is provisioned automatically on demand and released when idle.

Optimised for Analytics

Uses Photon, Databricks' vectorised query engine, to deliver fast performance on large-scale analytical workloads.

Why Not Just Use a Cluster?

Both clusters and SQL Warehouses can run SQL queries — but they are engineered for fundamentally different purposes. Choosing the wrong compute type leads to unnecessary cost, slower performance, and harder governance.

All-Purpose Cluster

- Designed for **development workloads** — notebooks, ETL pipelines, and ML experiments
- Supports mixed usage: Python, Scala, R, and SQL simultaneously
- Heavier startup times; not optimised for concurrent SQL queries
- Shared resources can cause contention across users

SQL Warehouse

- Designed exclusively for **analytics workloads** — dashboards, reports, and ad-hoc queries
- SQL-only interface, reducing risk of misuse for heavy ETL
- Photon-accelerated; optimised for high concurrency and fast response times
- Auto-scales per-query with predictable, auditable cost

SQL Warehouse Architecture

SQL Warehouses are designed to decouple analytical compute from your core data engineering infrastructure — ensuring that business users querying dashboards never compete with pipelines writing to Delta tables.

Separate Compute Layer

SQL Warehouses run on isolated compute, completely independent from your Spark clusters. This separation protects pipeline stability and ensures resource contention never degrades query performance for end users.

Auto-Scaling

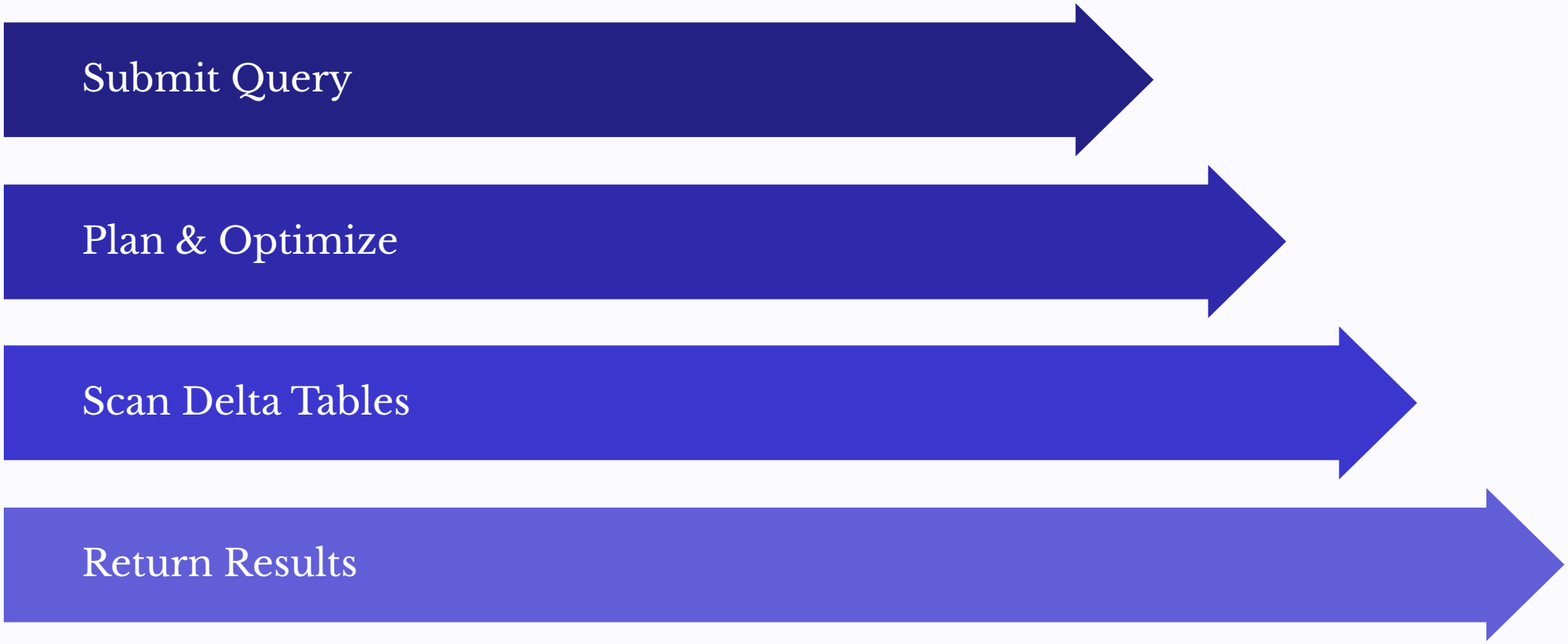
Warehouses automatically scale out to additional clusters when query concurrency increases, and scale back down during quiet periods — ensuring consistent performance without manual intervention or over-provisioning.

Query Optimisation

The Photon engine rewrites and optimises query plans at runtime, leveraging Delta table statistics, Z-ordering metadata, and vectorised execution to minimise data scanned and maximise throughput.

Query Execution Flow

Understanding how a query travels from user intent to result set helps you design performant workloads and diagnose bottlenecks. Every step is managed within the Databricks platform — no external query routing required.



At each stage, the SQL Warehouse applies query rewriting, partition pruning, and caching to minimise latency. Delta's transaction log ensures every query reads a consistent, correct snapshot of the data — even while pipelines are actively writing.

Cost Model

SQL Warehouses use a **consumption-based pricing model** — you are billed for the compute time actively used to execute queries, not for idle infrastructure. This makes cost directly traceable to business activity.

Pay Per Query / Compute

Costs are measured in **Databricks Units (DBUs)** per second of active execution. Idle time between queries does not accrue charges when auto-stop is configured correctly.

Auto Stop / Start

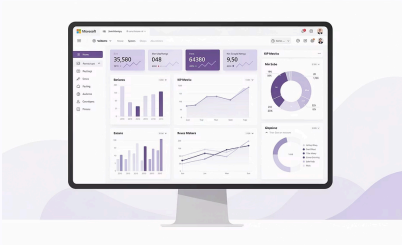
Warehouses automatically suspend after a configurable idle timeout (e.g., 10 minutes) and restart on the next incoming query — cold start latency is typically under 30 seconds for serverless tier.

Enterprise Governance

Cost attribution is straightforward: tag warehouses by team or product, monitor DBU consumption in the Databricks account console, and enforce spend limits through workspace policies.

BI & Integration Support

SQL Warehouses are the recommended connection point for all downstream BI tools and custom applications. They expose standard connectivity interfaces, making integration straightforward regardless of your existing toolchain.

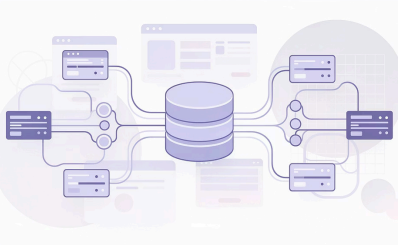


Power BI

Connect via the native Databricks connector in Power BI Desktop. Supports DirectQuery mode for live data access against Delta tables, ensuring dashboards always reflect the latest Gold layer state.

Tableau

Use the Databricks Tableau connector or standard JDBC driver. Tableau's Live connection mode routes queries directly to the SQL Warehouse, bypassing extract limitations for large datasets.



JDBC / ODBC

Any SQL-compatible application can connect via industry-standard JDBC or ODBC drivers. This covers custom applications, legacy reporting tools, and data science environments like R Studio or DBeaver.

Best Practices

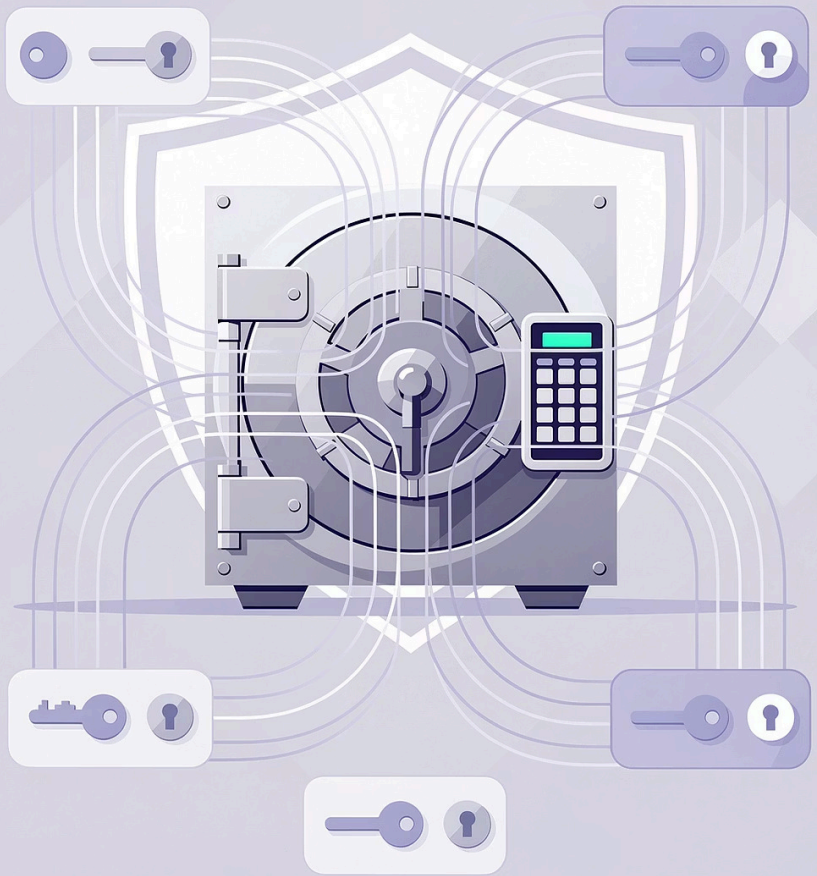
✅ Use SQL Warehouses For

- **Interactive dashboards** — Power BI, Tableau, and Databricks SQL dashboards with live Delta connections
- **Scheduled reporting** — automated SQL queries producing daily or hourly summary outputs
- **Ad-hoc analytics** — analysts running exploratory queries against Gold layer tables
- **High-concurrency workloads** — multiple users querying simultaneously without resource contention

❌ Avoid SQL Warehouses For

- **Heavy ETL pipelines** — use Delta Live Tables or Spark Jobs on all-purpose or job clusters instead
- **Complex transformations** — multi-step Python or Scala logic belongs in notebooks or DLT pipelines
- **ML model training** — use ML clusters with GPU or high-memory configurations

> Keeping ETL and analytics on separate compute tiers is the single most effective way to control cost and maintain pipeline reliability in production.



What's Next: Data Access Control

You now have a fully operational serving layer — Gold data flowing to BI tools and analysts via a performant, cost-efficient SQL Warehouse. The architecture is working. But one critical question remains unanswered.

> **"Who is allowed to see which data?"**
> Serving data at scale without access controls is a governance risk. In Day 7, we introduce Unity Catalogue — Databricks' unified governance layer for managing permissions, data lineage, and compliance across your entire lakehouse.

1

Complete

SQL Warehouse — serving Gold to the business

2

Up Next

Unity Catalogue — controlling who can access this data